



Learn Beyond

KPR Institute of Engineering and Technology

Autonomous, Affiliated to Anna University

Avinashi Road, Arasur, Coimbatore - 641 407

U21AD502 – APPLICATION DEVELOPMENT

III year B.Tech. Artificial Intelligence and Data Science

Mini Project Report

CAMPUS LOST & FOUND PORTAL

A Full-Stack Web Application using MongoDB, Express.js, React and Node.js

Submitted by

NAKUL T

Register No: **24AD068**

Academic Year: 2026 - 27

Abstract

Students regularly lose belongings on campus, such as ID cards, calculators, chargers, books and keys, and the only ways to report them today are scattered WhatsApp groups and notice boards. This project builds a **Campus Lost & Found Portal**, a web application where students can post items they have lost or found, browse every post in one place, and search by category and keyword. It uses the **MERN stack**: a **MongoDB** database modelled with **Mongoose**, a RESTful **Node.js/Express** API that supports GET, POST, PUT and DELETE, and a **React** single-page front end. The report presents the database schema, the API design, the React component structure with its state management, and the complete data flow from a user submitting a post until it appears in the listing.

1. Introduction

1.1 Problem Statement

A college wants a "Lost and Found" portal where students can post items they have lost or found, view all posts, and search by category (electronics, books, ID cards, etc.). The system must store the posts, show them to everyone, and let the author update a post (for example, mark it resolved once the item is returned) or delete it.

1.2 Objectives

1. Design a MongoDB schema for storing lost/found posts, with validation and indexes.
2. Design and build Node.js API routes (GET, POST, PUT, DELETE) to support the feature.
3. Design the React component structure for the posting form and the listing page, including state management.
4. Explain the complete data flow: frontend → backend → database → frontend.

1.3 Scope

- Create posts of type *lost* or *found*, with category, location, date, description and contact details.
- List posts newest-first, with filters for category, type and status, keyword search, and pagination.
- Mark posts as resolved or reopen them, and delete posts.
- A responsive layout that works on desktop and mobile browsers.

2. Technology Stack

Layer	Technology	Purpose
Database	MongoDB 8	Stores posts as JSON-like (BSON) documents in the <code>posts</code> collection
ODM	Mongoose 8	Schema definition, validation, indexes and query building
Backend	Node.js + Express 4	REST API server, middleware, routing and error handling
Frontend	React 18 + Vite 5	Component-based single-page user interface with hooks for state
Communication	HTTP + JSON (Fetch API)	Client-server data exchange; the Vite dev proxy forwards <code>/api</code> to Express
Tools	npm, VS Code, Chrome DevTools, curl, mongosh	Development, debugging and API/database testing

Table 1: Technology stack

3. System Architecture

The application follows a **three-tier architecture**. The React client talks to the Express server only through HTTP/JSON, and only the server talks to the database.

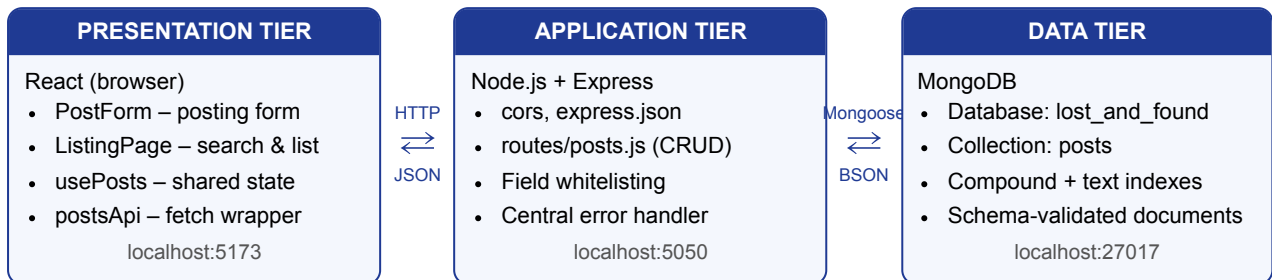


Fig. 1: Three-tier architecture of the portal

4. (a) MongoDB Schema Design

4.1 Design Decisions

- **Single posts collection** for both lost and found items, told apart by a `type` field. Both have the same fields, so "view all posts" is one query and lost/found is just another filter.
- **Category as an enum string**. Categories are few and fixed, so an enum rejects invalid values and avoids a separate collection and `$lookup` joins.
- **Embedded contact sub-document**. Contact details are always read together with the post and never queried alone, so embedding is the natural document-model choice.
- **date kept separate from createdAt**. One records when the item was lost or found, the other when it was posted.
- **status (open / resolved)** lets returned items be closed without deleting them, and **timestamps** maintain `createdAt/updatedAt` for newest-first sorting.

4.2 Collection Structure – posts

Field	Type	Required	Constraints / Notes
<code>_id</code>	ObjectId	auto	Primary key generated by MongoDB
<code>type</code>	String	Yes	enum: <code>lost</code> , <code>found</code>
<code>title</code>	String	Yes	3–100 characters, trimmed
<code>description</code>	String	Yes	Maximum 1000 characters
<code>category</code>	String	Yes	enum: electronics, books, id-cards, keys, bags, clothing, accessories, stationery, others
<code>location</code>	String	Yes	Where the item was lost/found, e.g. "Central Library, 2nd floor"
<code>date</code>	Date	Yes	Date lost/found; custom validator rejects future dates
<code>imageUrl</code>	String	No	Optional photo link
<code>contact</code>	Object	Yes	{ <code>name</code> (req.), <code>email</code> (req., regex-validated), <code>phone</code> (optional, 7–15 digits) }
<code>status</code>	String	auto	enum: <code>open</code> (default), <code>resolved</code>
<code>createdAt</code> , <code>updatedAt</code>	Date	auto	Maintained by Mongoose <code>timestamps</code> option

Table 2: Fields of the posts collection

4.3 Mongoose Schema (backend/models/Post.js)

```
const CATEGORIES = ['electronics', 'books', 'id-cards', 'keys', 'bags',
  'clothing', 'accessories', 'stationery', 'others'];

const contactSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true, maxlength: 60 },
  email: { type: String, required: true, lowercase: true,
    match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email'] },
  phone: { type: String, match: [/^[0-9+\-\s]{7,15}$/, 'Please enter a valid phone number'] },
}, { _id: false }); // embedded, no separate _id

const postSchema = new mongoose.Schema({
  type: { type: String, enum: ['lost', 'found'], required: true },
  title: { type: String, required: true, trim: true, minlength: 3, maxlength: 100 },
  description: { type: String, required: true, trim: true, maxlength: 1000 },
  category: { type: String, enum: CATEGORIES, required: true },
  location: { type: String, required: true, trim: true, maxlength: 120 },
  date: { type: Date, required: true,
    validate: { validator: v => v.getTime() <= Date.now() + 24*60*60*1000,
      message: 'Date cannot be in the future' } },
  imageUrl: { type: String, trim: true },
  contact: { type: contactSchema, required: true },
  status: { type: String, enum: ['open', 'resolved'], default: 'open' },
}, { timestamps: true }); // adds createdAt, updatedAt

module.exports = mongoose.model('Post', postSchema);
```

4.4 Indexes

```
// Compound index: serves "filter by category/type, newest first" without a collection scan
postSchema.index({ category: 1, type: 1, createdAt: -1 });

// Weighted text index over the searchable fields
postSchema.index({ title: 'text', description: 'text', location: 'text' },
  { weights: { title: 5, location: 2, description: 1 } });
```

Without an index, every category search has to scan the whole collection (**COLLSCAN**). The compound index lets MongoDB jump straight to the matching, already-sorted entries (**IXSCAN**), so the portal stays fast as posts pile up over many semesters.

4.5 Sample Stored Documents

Fig. 2 shows real documents in the `posts` collection, queried with `mongosh` after posts were created from the React form.

```
lost_and_found> db.posts.find({ category: "accessories" }, { title: 1, type: 1, location: 1, status: 1, createdAt: 1 })
[
  {
    _id: ObjectId('6ab9467497a5dd7b9b913243'),
    type: 'found',
    title: 'Silver Titan wristwatch',
    location: 'Library reading room',
    status: 'resolved',
    createdAt: ISODate('2026-09-27T16:38:12.641Z')
  },
  {
    _id: ObjectId('6ab9466ef0a2c404bab6bf86'),
    type: 'found',
    title: 'Boat Airdopes (white case)',
    location: 'Seminar Hall 1',
    status: 'open',
    createdAt: ISODate('2026-09-27T16:38:06.121Z')
  }
]
```

Fig. 2: Documents stored in MongoDB (mongosh query on the posts collection)

5. (b) Node.js API Routes

The Express server exposes a RESTful resource at `/api/posts`. Every response has the shape `{ success, data | message, errors? }`.

Method	Endpoint	Description	Success	Errors
GET	<code>/api/posts</code>	List posts, with filtering, keyword search and pagination	200	500
GET	<code>/api/posts/categories</code>	Categories with the number of open posts in each	200	500
GET	<code>/api/posts/:id</code>	Fetch a single post	200	400, 404
POST	<code>/api/posts</code>	Create a new lost/found post (status always starts as <i>open</i>)	201	400
PUT	<code>/api/posts/:id</code>	Update a post: edit details or mark resolved / reopen	200	400, 404
DELETE	<code>/api/posts/:id</code>	Delete a post	200	400, 404

Table 3: REST API routes

Query parameters for GET `/api/posts`: `category` (e.g. `electronics`), `type` (`lost/found`), `status` (`open/resolved`), `q` (case-insensitive search over title, description and location), `page` and `limit` (maximum 50).

5.1 Route Implementation (key parts of `routes/posts.js`)

```
router.get('/', async (req, res, next) => { // GET – list / filter / search
  const { category, type, status, q } = req.query;
  const filter = {};
  if (category && category !== 'all') filter.category = category;
  if (type && type !== 'all') filter.type = type;
  if (status && status !== 'all') filter.status = status;
  if (q) { const rx = new RegExp(escapeRegex(q.trim()), 'i');
    filter.$or = [{ title: rx }, { description: rx }, { location: rx }]; }
  const [posts, total] = await Promise.all([
    Post.find(filter).sort({ createdAt: -1 }).skip((page - 1) * limit).limit(limit).lean(),
    Post.countDocuments(filter) ]);
  res.json({ success: true, data: posts, pagination: { page, limit, total, pages } });
});

router.post('/', async (req, res, next) => { // POST – create
  const data = pick(req.body, EDITABLE_FIELDS); delete data.status;
  const post = await Post.create(data); // schema validation runs here
  res.status(201).json({ success: true, data: post });
});

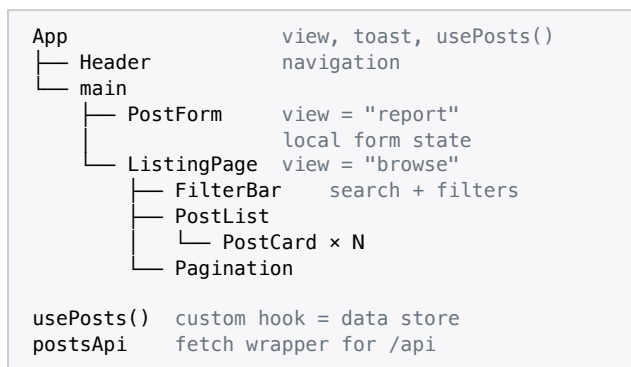
router.put('/:id', async (req, res, next) => { // PUT – update (404 if not found)
  const post = await Post.findByIdAndUpdate(req.params.id, pick(req.body, EDITABLE_FIELDS),
    { new: true, runValidators: true });
  res.json({ success: true, data: post });
});

router.delete('/:id', async (req, res, next) => { // DELETE – remove (404 if not found)
  await Post.findByIdAndDelete(req.params.id); res.json({ success: true, message: 'Post deleted' });
}); // try/catch → next(err) and 404 checks omitted for brevity
```

5.2 Validation, Error Handling and Security

- errorHandler** middleware turns a Mongoose `ValidationError` into a 400 with per-field messages (e.g. `{"errors": {"title": "Title must be at least 3 characters", "contact.email": "Please enter a valid email"}}`), a `CastError` (bad id) into a 400, and anything else into a 500.
- Field whitelisting** (`pick`) stops clients from overwriting `_id` or `createdAt`.
- Search text is **regex-escaped**, `limit` is capped at 50, and `runValidators` applies the schema rules to updates as well.

6. (c) React Component Structure & State Management



The UI is split into small components with a single responsibility each. **App** is the root: it switches between the posting form and the listing, and owns the shared post data by calling the **usePosts** custom hook. Data flows **down as props** and user actions flow **up as callbacks**, React's one-way data flow.

The listing and the form never call `fetch` themselves. All HTTP calls go through **postsApi**, and all server state lives in **usePosts**, which keeps the components purely presentational. **useState** plus a custom hook is enough for an application this size (**Redux would be over-engineering**); if the app grows, the hook can move into a React Context provider without changing the components.

Fig. 3: React component tree

Component	Local state	Responsibility
App	view, toast	Root; switches views; calls <code>usePosts()</code> ; shows notifications
Header	—	"Browse Items" / "Report an Item" navigation
PostForm	form, errors, submitting, serverError	Controlled inputs, client-side validation, builds the JSON payload, shows server errors
ListingPage	—	Lays out filters, list and pagination; handles resolve/delete
FilterBar	—	Search box, type/status dropdowns, category chips
PostList	—	Renders the grid of cards or an empty-state message
PostCard	—	Shows one post with LOST/FOUND and category badges

Table 4: Component responsibilities

6.1 State Management Strategy

State is kept as close as possible to where it is used, and is lifted up only when siblings need to share it:

State	Lives in	Reason
Form fields, validation errors, submitting flag	PostForm (useState)	Only the form needs them; resetting the form doesn't affect the rest of the app
posts, filters, pagination, loading, error	usePosts hook in App	Shared: a post created in the form must appear in the listing
Current view, toast message	App (useState)	Global UI state

```

export function usePosts() {
  const [posts, setPosts] = useState([]);
  const [filters, setFilters] = useState({ category: 'all', type: 'all', status: 'open', q: '' });
  const fetchPosts = useCallback(async () => {
    setPosts((await postsApi.list({ ...filters, page })).data); }, [filters, page]);
  useEffect(() => { const t = setTimeout(fetchPosts, 300); // debounce search typing
    return () => clearTimeout(t); }, [fetchPosts]);
  const createPost = async (data) => { const res = await postsApi.create(data);
    setPosts(prev => [res.data, ...prev]); fetchPosts(); }; // show instantly, then re-sync
  return { posts, filters, updateFilters, createPost, updatePost, deletePost, ... };
}
  
```

7. (d) Complete Data Flow

Fig. 4 traces one post from the moment the user clicks **Submit** until it appears in the listing.

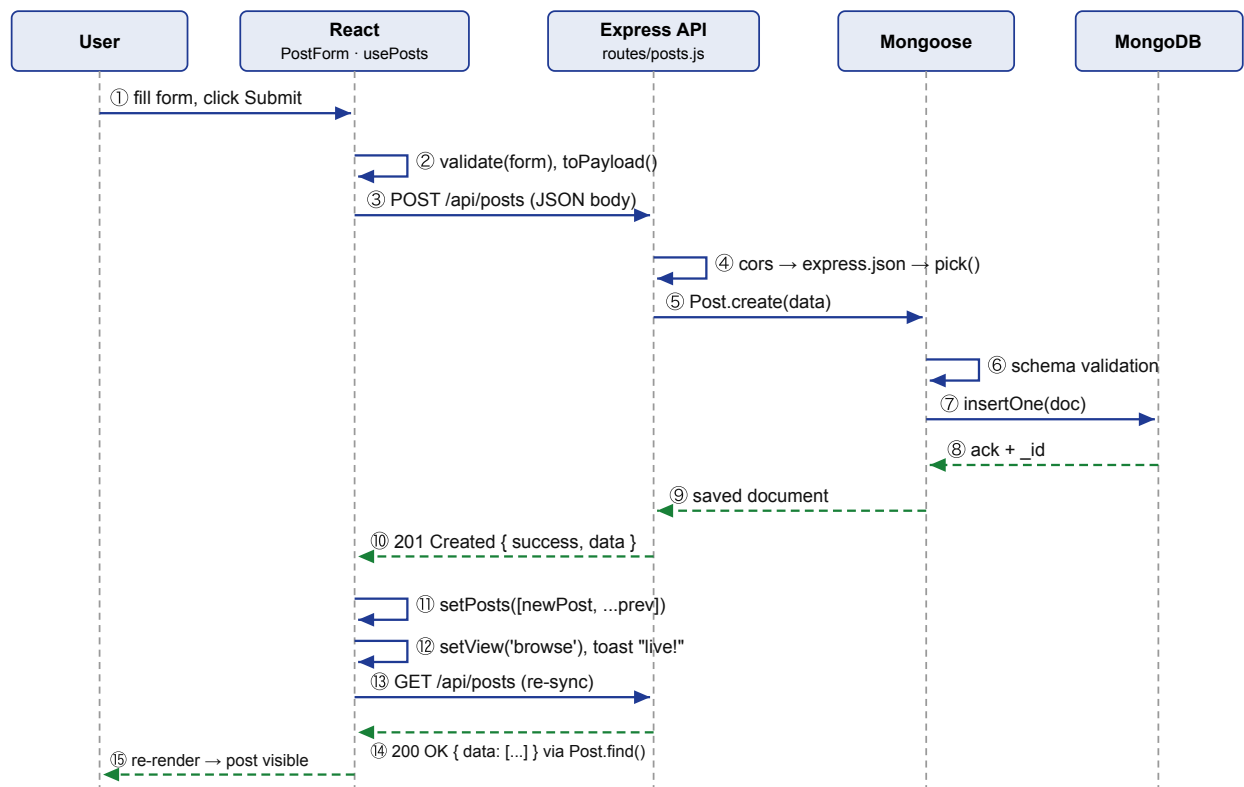


Fig. 4: Sequence diagram for creating a post (solid = request, dashed = response)

7.1 Step-by-Step Explanation

Frontend (React). The user opens *Report an Item*, and App renders `PostForm`. Each input is a **controlled component**: its value comes from the `form` state and every keystroke calls `setForm`. On Submit, `handleSubmit` calls `e.preventDefault()` and runs `validate(form)`; if a field is invalid, the errors are shown and no request is sent. `toPayload()` turns the flat form fields into the nested JSON the API expects (for example, `contactName` → `contact.name`), then `usePosts.createPost` calls `postsApi.create()`, which sends `fetch('/api/posts', {method: 'POST', body: JSON.stringify(data)})`.

Backend (Express). The request reaches Express on port 5050. The `cors` middleware checks the origin and `express.json()` parses the body into `req.body`. The router matches `POST /api/posts`, whitelists the allowed fields, removes status so every new post starts *open*, and calls `Post.create()`.

Database (Mongoose → MongoDB). Mongoose validates the document against the schema: required fields, enums, lengths, email format and no future dates. If it fails, the error handler returns **400** with messages per field, which `PostForm` shows under the matching inputs. If it passes, Mongoose applies the defaults and timestamps, MongoDB runs `insertOne` (generating the `_id` and updating the indexes), and Express replies **201 Created** with the saved document.

Back to the frontend. `createPost` prepends the new post to the `posts` state so it appears immediately, then calls `fetchPosts()` to re-sync the counts and pagination with the database. App switches back to the listing and shows "Your post is live!". Because the state changed, React re-renders `ListingPage` → `PostList` → `PostCard`, and the post appears at the top. **Search** follows the same path: changing a filter updates the `filters` state, `useEffect` waits 300 ms (debounce) and calls `GET /api/posts?category=...&q=...`, and the returned list replaces `posts`. **Resolve** and **Delete** call PUT and DELETE `/api/posts/:id` and update the state the same way.

8. Output Screenshots

The screenshots below were captured from the running application, backed by a local MongoDB database with sample data.

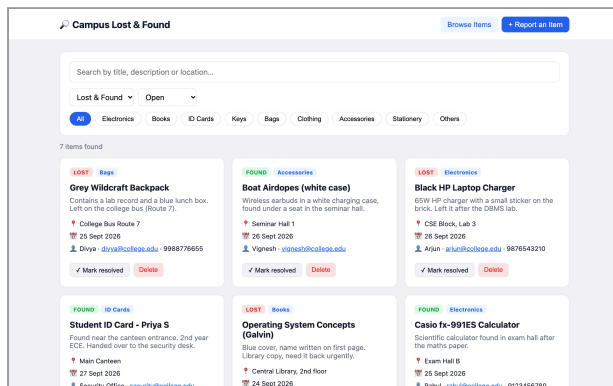


Fig. 5: Listing page – all open posts, newest first

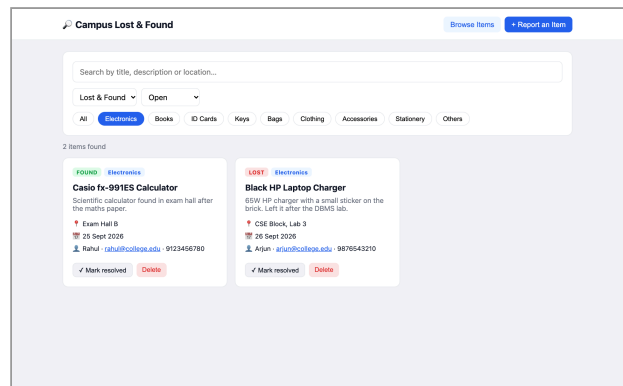


Fig. 6: Filtered by category "Electronics"

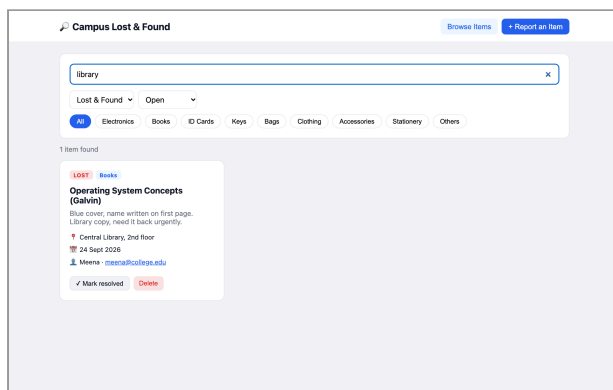


Fig. 7: Keyword search for "library"

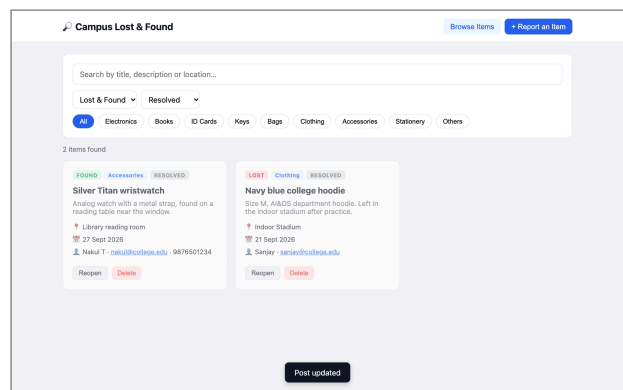


Fig. 8: Status filter showing resolved posts

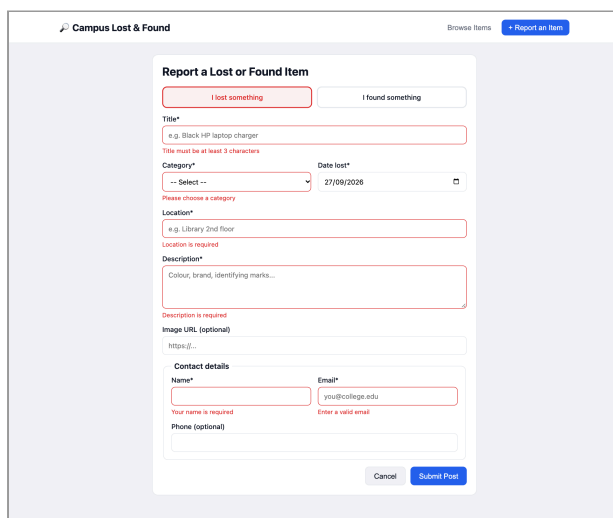


Fig. 9: Posting form – client-side validation errors

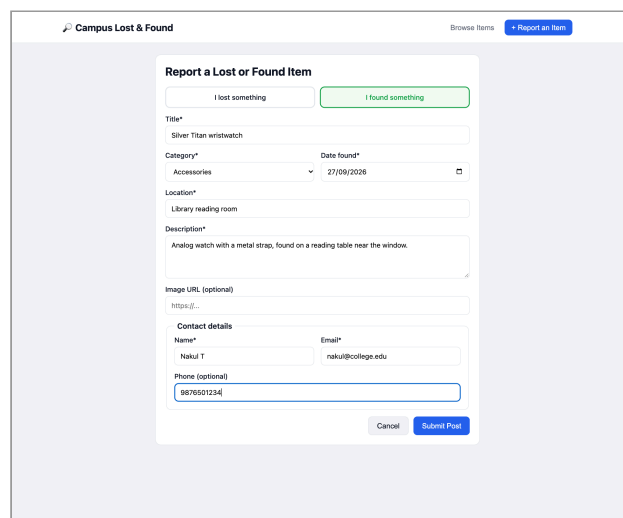


Fig. 10: Posting form filled in for a found item

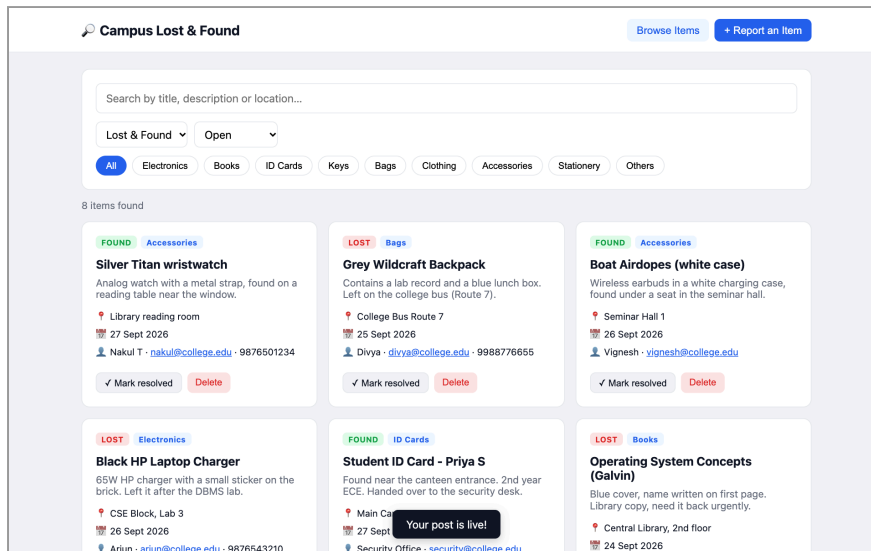


Fig. 11: After submitting, the new post appears at the top with a confirmation toast

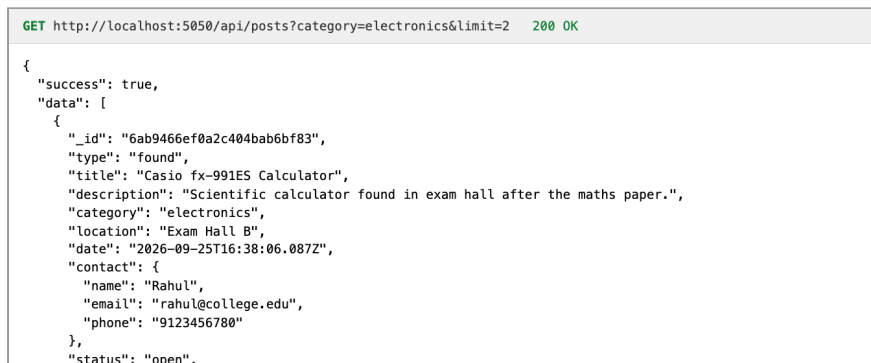


Fig. 12: JSON response of GET /api/posts?category=electronics

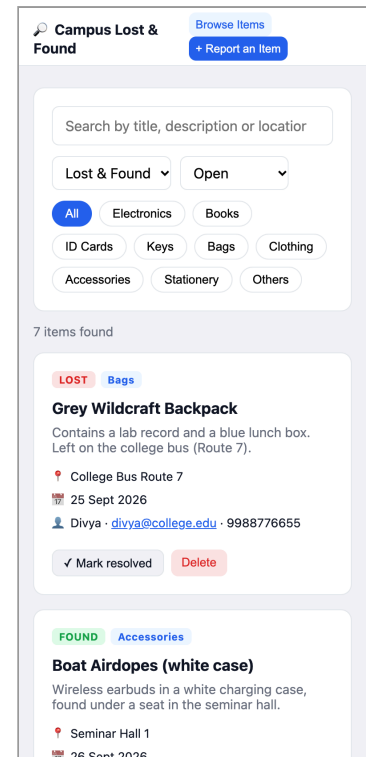


Fig. 13: Responsive layout on a mobile screen

9. Testing

The API was tested with `curl`, and the UI flows were exercised in the browser. All test cases passed.

#	Test case	Input	Expected result	Status
1	Filter by category	GET /posts? category=electronics	Only electronics posts returned	Pass
2	Partial keyword search	GET /posts?q=calc	"Casio fx-991ES Calculator" returned	Pass
3	Create valid post	POST /posts with status:"resolved"	201; status forced to "open"	Pass
4	Create invalid post	Bad type, short title, bad email	400 with 8 field-level messages	Pass
5	Mark resolved	PUT /posts/:id {status:"resolved"}	200; updated document returned	Pass
6	Delete / fetch deleted	DELETE then GET /posts/:id	200 "Post deleted", then 404	Pass
7	Form validation (UI)	Submit empty form	Errors under each required field; no request sent	Pass
8	End-to-end post (UI)	Fill form, Submit	Post at top of listing, toast shown, stored in MongoDB	Pass

Table 5: Test cases and results

10. Project Structure & Execution

```

lost-and-found-portal/
├── backend/
│   ├── models/Post.js      schema + indexes
│   ├── routes/posts.js     CRUD routes
│   ├── middleware/errorHandler.js
│   ├── server.js           app + DB connect
│   └── seed.js             sample data
├── frontend/src/
│   ├── api/postsApi.js     fetch wrapper
│   ├── hooks/usePosts.js   state hook
│   ├── components/        6 components
│   ├── App.jsx, main.jsx
└── report/                this report
  
```

```

# 1. Backend → http://localhost:5050
cd backend
cp .env.example .env
npm install
npm run seed      # sample posts
npm start

# 2. Frontend → http://localhost:5173
cd frontend
npm install
npm run dev
  
```

Source code: <https://github.com/nakultt/lost-and-found-portal>

11. Future Enhancements

1. **Authentication:** college-email login (JWT or Google SSO), so that only the author of a post can edit or delete it.
2. **Image upload:** direct photo upload using Multer with Cloudinary or S3, instead of pasting an image URL.
3. **Smart matching:** when a "found" item is posted, suggest "lost" posts with the same category and similar keywords, and email their owners.
4. **Admin panel:** let the security office moderate posts and archive resolved ones; a TTL index could auto-expire old posts.
5. **Real-time updates:** use Socket.IO so new posts appear without refreshing the page.

12. Conclusion

The Campus Lost & Found Portal meets all four requirements of the problem statement. **(a)** A single, validated and indexed MongoDB `posts` collection, with an embedded contact sub-document and enum-based categories, stores all lost and found items efficiently. **(b)** A RESTful Express API provides GET (list, filter, search, paginate, fetch one), POST, PUT and DELETE routes, with proper HTTP status codes and central error handling. **(c)** A modular React component tree keeps form state local to `PostForm` and shared post data in the `usePosts` custom hook, following one-way data flow. **(d)** The end-to-end data flow runs from a controlled form, through `fetch`, Express middleware, Mongoose validation and a MongoDB insert, back to a JSON response and a React re-render.

The project shows how the MERN stack's single language (JavaScript) and single data format (JSON) across every tier make it quick to build a practical tool that solves a real campus problem.

13. References

1. MongoDB Documentation – Data Modeling and Indexes. <https://www.mongodb.com/docs/manual/>
2. Mongoose Documentation – Schemas, Validation and Models. <https://mongoosejs.com/docs/>
3. Express.js Documentation – Routing and Error Handling. <https://expressjs.com/>
4. React Documentation – Managing State and Reusing Logic with Custom Hooks. <https://react.dev/learn>
5. Vite Documentation – Server Options (proxy). <https://vitejs.dev/config/>